Universidad de San Andrés

Ingeniería en Inteligencia Artificial

I206 – Inferencia y Estimación

Trabajo Práctico N.º 2

Compresión de imágenes sin pérdida mediante codificación de Huffman

Codificación píxel a píxel, codificación diferencial e información mutua

Grupo 1

Bautista Alsina	Legajo 370833
Santos Bunge	Legajo 370842
Hans Dietrich	Legajo 370913

Profesores: Copa, Débora; Carducci, Leonardo; García Gulisano Pau

Primavera 2026

Resumen

En este trabajo se comprimen tres imágenes en escala de grises con el código de Huffman y se compara el resultado de codificar directamente cada píxel con el de codificar la diferencia entre píxeles vecinos. Para cada imagen se estima la masa de probabilidad de las intensidades y de los residuos, se calculan sus entropías y se verifica que la longitud media del código de Huffman queda entre la entropía y la entropía más un bit. La codificación diferencial más que duplica la compresión en la imagen más suave y la mejora en la segunda, pero resulta peor que la codificación directa en una textura de arena. La información mutua entre píxeles vecinos explica estas diferencias: mide la redundancia espacial que el esquema diferencial puede aprovechar, y es casi nula en la textura. Por último se implementan dos programas de línea de comando que comprimen una imagen a un archivo binario y la recuperan sin pérdida.

1. Introducción

Una imagen en escala de grises de 8 bits es una matriz de enteros entre 0 y 255, y guardarla "tal cual" cuesta 8 bits por píxel sin importar qué tan frecuente sea cada intensidad. La teoría de la información dice que eso es un desperdicio siempre que algunas intensidades sean más comunes que otras: un código que asigne palabras cortas a los símbolos frecuentes y largas a los raros necesita, en promedio, menos bits por símbolo. La entropía de la fuente marca cuánto se puede ahorrar como máximo, y el algoritmo de Huffman construye el código libre de prefijo que más se acerca a ese límite.

Hay, sin embargo, una segunda fuente de ahorro que la codificación símbolo a símbolo no ve. En una imagen los píxeles vecinos suelen parecerse, así que la diferencia entre un píxel y el anterior es casi siempre pequeña y, por lo tanto, mucho más predecible que el valor absoluto del píxel. Codificar esas diferencias (residuos) en lugar de las intensidades cambia la fuente por otra de menor entropía, sin perder información, porque la imagen se recupera sumando los residuos.

Los objetivos del trabajo son: (i) estimar empíricamente la masa de probabilidad de las intensidades de cada imagen, calcular su entropía y comprimir con Huffman, verificando la cota teórica sobre la longitud media del código; (ii) repetir el análisis sobre los residuos entre píxeles vecinos y cuantificar la mejora; (iii) estimar la información mutua entre píxeles vecinos y usarla para explicar las diferencias entre imágenes y entre esquemas; y (iv) implementar un compresor y un descompresor de línea de comando basados en Huffman sobre residuos.

2. Marco teórico y metodología

Entropía y código de Huffman. Sea X una fuente discreta con masa de probabilidad $p(x)$. Su entropía en bits, $H(X) = -\sum_x p(x) \log_2 p(x)$, es la cota inferior de la longitud media de cualquier código sin pérdida [4, 6]. Un código asigna a cada símbolo una palabra de $l(x)$ bits y su longitud media es $L = \sum_x p(x)\, l(x)$. El algoritmo de Huffman [5] construye el código libre de prefijo de menor L para una pmf dada, y cumple $H(X) \le L < H(X) + 1$: no alcanza exactamente la entropía porque las longitudes son enteras, pero la redundancia $L - H(X)$ es menor que un bit. Con una fuente de M símbolos usados vale además $H(X) \le \log_2 M$, con igualdad solo si son equiprobables.

Estimación empírica. Tratamos cada píxel como una realización de la fuente X y estimamos la pmf por frecuencia relativa, $\hat{p}(x) = n_x/N$, donde n_x es la cantidad de píxeles con intensidad x y N el total de píxeles. Es el histograma de 256 bins normalizado y el estimador de máxima verosimilitud de la pmf [2]. Todas las entropías del trabajo se calculan sobre estas estimaciones, sumando solo sobre los símbolos con $\hat{p}(x) > 0$.

Codificación diferencial. Recorremos la imagen en *serpentina*: la fila 0 de izquierda a derecha, la fila 1 de derecha a izquierda, y así sucesivamente. De este modo la imagen de $M \times N$ píxeles queda como una única secuencia $x_1, \ldots, x_{MN}$ en la que cada píxel es vecino espacial del anterior, y basta con guardar un único píxel inicial x_1 con sus 8 bits. El resto se representa con los residuos $y_k = x_k - x_{k-1}$, que toman valores en $\{-255, \ldots, 255\}$, y la imagen se reconstruye de forma exacta con $x_k = x_{k-1} + y_k$, es decir, con la suma acumulada de los residuos a partir de x_1. Elegimos este recorrido frente al de guardar el primer píxel de cada fila porque el costo fijo baja de $8M$ bits a 8 y porque deja toda la imagen como una sola fuente Y. La contrapartida es que un error en un residuo se propagaría a todos los píxeles siguientes y no solo al resto de la fila.

Información mutua. Para cuantificar la dependencia entre vecinos llamamos X' al píxel anterior y X al actual, tomando exactamente los mismos pares (x_{k-1}, x_k) que generan los residuos. La información mutua

$$I(X; X') = \sum_{x,x'} \hat{p}(x, x') \log_2 \frac{\hat{p}(x, x')}{\hat{p}(x)\,\hat{p}(x')} = H(X) + H(X') - H(X, X')$$

mide cuántos bits de incerteza sobre un píxel se eliminan al conocer su vecino [6, 3]. La pmf conjunta se estima con el histograma bidimensional de 256×256 bins. Como $H(X \mid X') = H(X) - I(X; X')$ es lo mínimo que hace falta codificar una vez conocido el vecino, y el residuo Y es una función de (X, X'), se cumple $H(Y) \geq H(X \mid X')$: la información mutua acota la ganancia que cualquier esquema basado en el vecino puede obtener. Las marginales de esta sección se calculan sobre las secuencias de los pares, que tienen $MN - 1$ muestras (X' omite el último píxel y X el primero), para que los tres términos de la identidad usen las mismas muestras. Difieren de la $H(X)$ de la Sección 3 en un solo píxel y coinciden en todas las cifras mostradas.

Implementación. Las tres imágenes provistas (Figura 1) vienen en color; las convertimos a escala de grises de 8 bits antes de todo el análisis, así que "sin pérdida" se refiere siempre a esa versión. Los ejercicios 1 a 3 están en un único cuaderno que reutiliza las mismas funciones de estimación de pmf, entropía y compresión para intensidades y para residuos; las funciones de construcción y decodificación del código de Huffman son las provistas por la cátedra [1]. En cada compresión se decodifica la cadena de bits y se verifica que la imagen reconstruida coincide píxel a píxel con la original. Las cifras citadas en este informe son las que imprime el cuaderno.

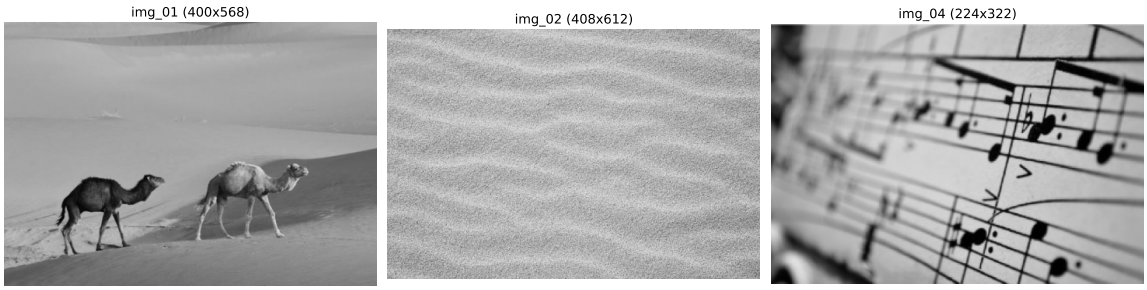


Figura 1: Las tres imágenes en escala de grises, con sus dimensiones. `img_01`: dunas con camellos, grandes regiones suaves. `img_02`: textura de arena de grano fino. `img_04`: partitura, fondo uniforme con trazos oscuros.

3. Ejercicio 1: codificación píxel a píxel

Masa de probabilidad y entropía (inciso a). La fila superior de la Figura 2 muestra $\hat{p}(x)$ para cada imagen y la Tabla 1 resume las entropías. Las tres imágenes usan una cantidad parecida de niveles de gris (entre 205 y 228 de los 256 posibles), de modo que la cota $\log_2 M \approx 7.7$ bits es casi la misma para todas; la diferencia de entropía está en cómo se reparte la masa dentro de ese rango. `img_01` tiene un pico muy marcado alrededor de 165 (el tono de la arena de las dunas) y

otro menor en 100, con poca masa fuera de ellos, y por eso su entropía es la más baja. `img_02` usa menos niveles pero los reparte de forma acampanada y bastante pareja entre 120 y 240. `img_04` tiene la pmf más plana: una meseta entre 0 y 150 correspondiente a la tinta y las sombras, y una zona densa entre 150 y 220 para el fondo, lo que cubre casi todo el rango y da la entropía más alta.

Tabla 1: Codificación píxel a píxel. Niveles usados, entropía, longitud media teórica L del código de Huffman, bits promedio medidos sobre la cadena comprimida y tasa de compresión respecto de 8 bits.

Imagen	Píxeles	Niveles	$H(X)$ [bits]	L [bits]	Bits/píxel medidos	$8/L$
`img_01`	227 200	224	6.281	6.310	6.310	1.27
`img_02`	249 696	205	6.783	6.809	6.809	1.17
`img_04`	72 128	228	7.138	7.165	7.165	1.12

Compresión con Huffman (inciso b). Con la pmf estimada construimos el código de Huffman, codificamos cada píxel y concatenamos las palabras. Los bits promedio por píxel de la cadena resultante coinciden exactamente con la longitud media teórica L (Tabla 1), como corresponde: al reemplazar $\hat{p}(x) = n_x/N$ en $L = \sum_x \hat{p}(x)\, l(x)$ se obtiene $\frac{1}{N}\sum_x n_x\, l(x)$, que es la cantidad total de bits dividida por N. La comparación sirve como verificación de consistencia entre la implementación y la teoría. En las tres imágenes se cumple $H(X) \le L < H(X) + 1$ y la redundancia $L - H(X)$ es de apenas 0.03 bits, muy por debajo del bit de margen que garantiza la cota.

La compresión frente a los 8 bits por píxel es modesta: la tasa $8/L$ va de 1.12 a 1.27, es decir, un ahorro de entre el 11 % y el 21 %. Como Huffman ya está prácticamente en el límite $H(X)$, ningún código símbolo a símbolo puede hacerlo mucho mejor sobre esta fuente. Para comprimir más hace falta cambiar la fuente, que es lo que hace la codificación diferencial.

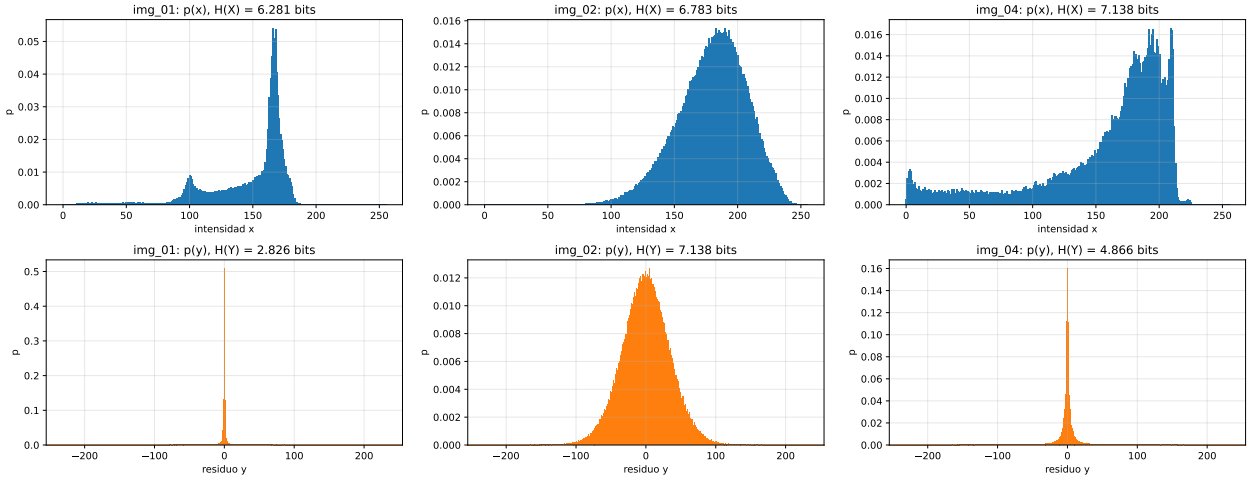


Figura 2: Fila superior: pmf empírica de las intensidades $\hat{p}(x)$ de cada imagen. Fila inferior: pmf empírica de los residuos $\hat{p}(y)$ sobre el alfabeto $\{-255, \ldots, 255\}$. Notar las escalas verticales: en `img_01` el residuo nulo concentra la mitad de la masa.

4. Ejercicio 2: codificación diferencial

4.1. Masa de probabilidad y entropía de los residuos (inciso a)

La fila inferior de la Figura 2 y la Figura 3 muestran $\hat{p}(y)$, y la Tabla 2 compara las entropías. El comportamiento no es el mismo en las tres imágenes.

Tabla 2: Entropía de las intensidades y de los residuos, cantidad de residuos distintos y fracción de residuos con $|y| \leq 5$.

Imagen	$H(X)$ [bits]	$H(Y)$ [bits]	$H(X) - H(Y)$	Residuos distintos	$\hat{\mathbb{P}}(\lvert Y\rvert \leq 5)$
`img_01`	6.281	2.826	3.455	230	0.925
`img_02`	6.783	7.138	−0.355	303	0.134
`img_04`	7.138	4.866	2.272	311	0.719

En `img_01` y `img_04` la pmf de los residuos es un pico angosto en $y = 0$ que decae rápido: el 92 % y el 72 % de los residuos, respectivamente, caen en $|y| \leq 5$. Por eso $H(Y)$ es bastante menor que $H(X)$, aun cuando el alfabeto de Y (511 símbolos posibles, de los cuales aparecen entre 230 y 311) es el doble de grande que el de X. Es la redundancia espacial: como los píxeles vecinos tienen intensidades parecidas, su diferencia es pequeña y mucho más predecible que el valor absoluto.

En `img_02` ocurre lo contrario: $H(Y)$ supera a $H(X)$. Solo el 13 % de los residuos cae en $|y| \leq 5$ y $\hat{p}(y)$ es una campana ancha, de desvío comparable al de las intensidades. A escala de un píxel la arena se comporta como ruido: la intensidad de un píxel aporta poca información sobre la del vecino, y restar dos variables casi independientes no reduce la variabilidad sino que la amplifica, porque la diferencia tiene mayor dispersión que cada una por separado. La codificación diferencial solo ayuda cuando existe dependencia entre vecinos, lo que se cuantifica en el inciso (c).

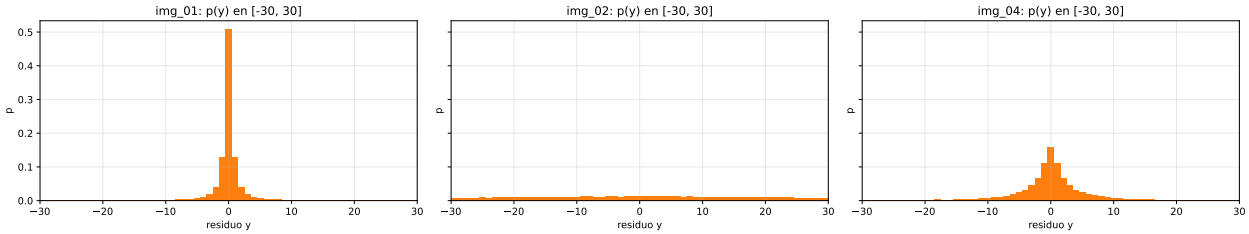


Figura 3: Detalle de $\hat{p}(y)$ en $[-30, 30]$. En `img_01` y `img_04` casi toda la masa está en unos pocos residuos alrededor de cero; en `img_02` la pmf es ancha incluso en esta ventana.

4.2. Compresión de los residuos (inciso b)

Aplicamos el mismo procedimiento del Ejercicio 1 sobre la fuente Y, con alfabeto $\{-255, \ldots, 255\}$. La Tabla 3 muestra que los bits promedio medidos vuelven a coincidir con la longitud media teórica L_Y y que se cumple $H(Y) \leq L_Y < H(Y) + 1$ en las tres imágenes. Para comparar de manera justa con el Ejercicio 1 contamos también los 8 bits del píxel inicial, que sobre toda la imagen son despreciables: el costo total por píxel coincide con L_Y hasta la cuarta cifra decimal.

Tabla 3: Codificación de los residuos con Huffman y comparación con la codificación píxel a píxel. La tasa se calcula respecto de los 8 bits por píxel de la representación directa.

Imagen	$H(Y)$	L_Y	Bits/píxel medidos	L_X (Ej. 1)	Tasa píxel a píxel	Tasa residuos
`img_01`	2.826	2.840	2.840	6.310	1.27	2.82
`img_02`	7.138	7.179	7.179	6.809	1.17	1.11
`img_04`	4.866	4.894	4.894	7.165	1.12	1.63

La Figura 4 resume los dos esquemas. En `img_01` la tasa de compresión pasa de 1.27 a 2.82 y en `img_04` de 1.12 a 1.63: como $\hat{p}(y)$ está concentrada en pocos símbolos, Huffman asigna palabras de 1 a 3 bits a los residuos cercanos a cero, que son la gran mayoría. En `img_02` la codificación de residuos es peor que la píxel a píxel ($L_Y = 7.18$ frente a $L_X = 6.81$), aunque sigue por debajo de 8 bits porque la pmf de los residuos conserva cierta concentración alrededor de cero. En todos los casos Huffman queda a menos de 0.05 bits de la entropía de la fuente que codifica: lo que cambia entre los dos esquemas no es el codificador sino la fuente.

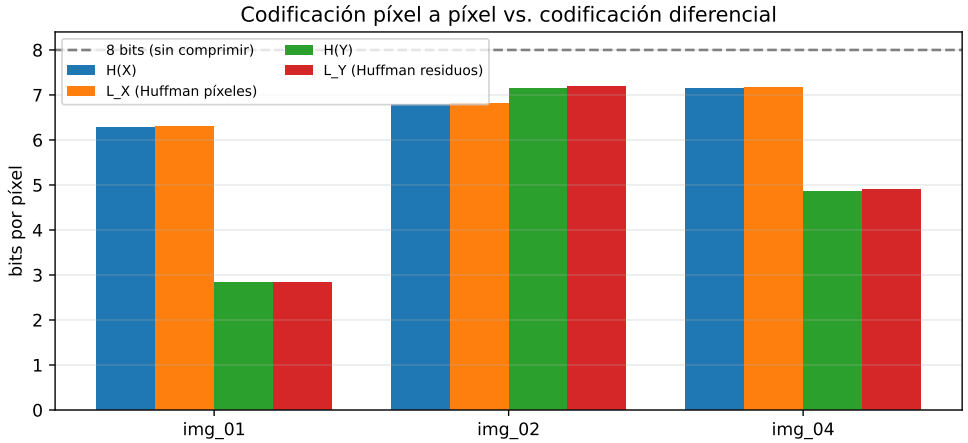


Figura 4: Entropía y longitud media del código de Huffman para las intensidades ($H(X)$, L_X) y para los residuos ($H(Y)$, L_Y), frente a los 8 bits de la representación directa.

4.3. Información mutua entre píxeles vecinos (inciso c)

La Figura 5 muestra la pmf conjunta $\hat{p}(x, x')$ de cada imagen en escala logarítmica (sin el logaritmo solo se vería la diagonal, porque la masa abarca varios órdenes de magnitud), y la Tabla 4 resume las cantidades de información. La diagonal $x = x'$ corresponde al residuo nulo, y la distancia a la diagonal es el valor del residuo, así que cada gráfico es a la vez una foto de la dependencia entre vecinos y de la distribución de residuos. Si X y X' fueran independientes, el mapa sería el producto de las dos marginales: una mancha sin estructura diagonal.

Tabla 4: Información mutua empírica entre píxeles vecinos y cantidades relacionadas, en bits.

Imagen	$H(X)$	$H(X, X')$	$I(X; X')$	$I(X; X')/H(X)$	$H(X \mid X')$	$H(Y)$	$H(X) - H(Y)$
`img_01`	6.281	8.840	3.722	0.59	2.559	2.826	3.455
`img_02`	6.783	13.450	0.116	0.02	6.667	7.138	−0.355
`img_04`	7.138	11.496	2.779	0.39	4.358	4.866	2.272

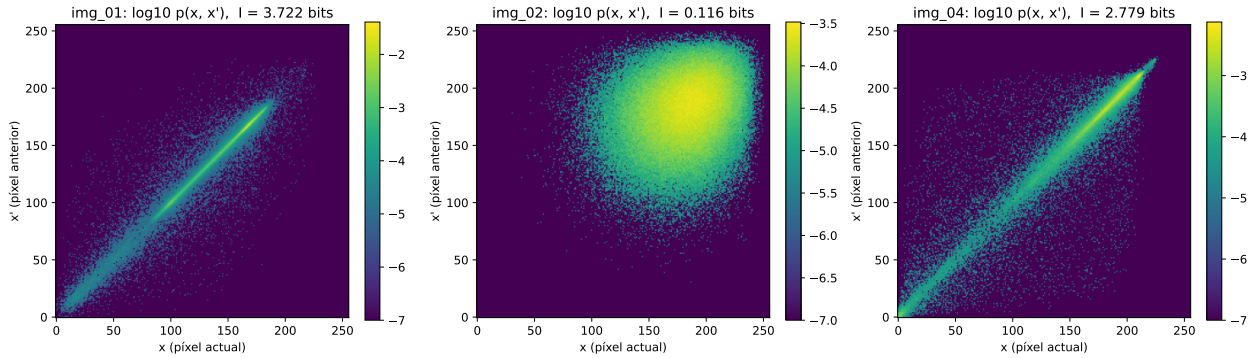


Figura 5: Pmf conjunta $\hat{p}(x, x')$ del píxel actual x y su vecino anterior x', en escala $\log_{10}$. Las celdas sin ocurrencias se muestran en el color más oscuro.

En `img_01` la masa se concentra en una diagonal fina y brillante, más intensa entre 100 y 190 (los tonos de la arena); los pocos puntos lejos de la diagonal son los bordes de los camellos. Conocer el vecino elimina el 59 % de la incerteza del píxel. En `img_04` la diagonal también es nítida y se extiende por todo el rango, con dos zonas densas: cerca de 0 a 30 (tinta seguida de tinta) y alrededor de 200 a 230 (fondo seguido de fondo); la nube alrededor es más ancha porque los bordes de las notas generan muchos pares con valores muy distintos, y $I(X; X')$ es algo menor. En `img_02` no hay diagonal: la pmf conjunta es una mancha difusa centrada en $(180, 180)$, justo la forma del producto de las marginales, y $I(X; X')$ es apenas 0.12 bits.

En las tres imágenes se verifica $H(X) - H(Y) \leq I(X; X')$, como corresponde a $H(Y) \geq H(X \mid X')$. En las imágenes suaves los residuos capturan casi toda la información mutua: $H(Y)$ queda a 0.27 y 0.51 bits de $H(X \mid X')$. En `img_02` la información mutua es pequeña pero positiva mientras que $H(X) - H(Y)$ es negativa: hay algo de dependencia, pero la resta $X - X'$ es una manera pobre de aprovecharla cuando la correlación es débil.

5. Ejercicio 3: interpretación de los resultados

(a) Diferencias de entropía entre imágenes. $H(X)$ depende únicamente de la forma de la pmf de intensidades: es máxima cuando los niveles usados son equiprobables y disminuye cuanto más concentrada está la masa en pocos niveles. Como las tres imágenes usan una cantidad similar de niveles, la diferencia no está en el rango sino en cuán repartida está la masa dentro de él. `img_01` tiene la entropía más baja porque la mayor parte de la imagen es arena de un tono similar y la pmf tiene un pico pronunciado; `img_02` reparte su masa de manera bastante pareja en una campana; `img_04` combina dos poblaciones bien separadas (fondo claro y tinta oscura) con transiciones entre ambas, lo que cubre casi todo el rango con una pmf relativamente plana y da la entropía más alta. La entropía refleja el contraste y la variedad tonal de la imagen, no su textura ni su estructura espacial, que la pmf de intensidades ignora por completo. Un ejemplo claro es `img_02`: tiene mucha variabilidad (entropía alta) pero, como se ve en el inciso siguiente, casi nada de ella es redundancia aprovechable.

(b) Qué representa la información mutua. $I(X; X')$ es la redundancia espacial de la imagen: la cantidad de bits de incerteza sobre un píxel que desaparecen al conocer su vecino. Para la codificación diferencial es la cantidad máxima de bits por píxel que cualquier esquema basado en el vecino podría ahorrar, porque lo mínimo que hay que codificar una vez conocido X' es $H(X \mid X') = H(X) - I(X; X')$, y los residuos cumplen $H(Y) \geq H(X \mid X')$. Las diferencias entre imágenes se deben a cuán parecidos son los píxeles vecinos, es decir, a la suavidad de la imagen a escala de un píxel: `img_01` es la más suave (regiones de dunas con variación gradual) y tiene la mayor $I(X; X')$; `img_04` tiene regiones uniformes pero también muchos bordes de tinta que generan pares muy distintos, y su $I(X; X')$ es menor; `img_02` es una textura de grano fino en la que la intensidad de un píxel casi no dice nada sobre la del vecino, y su $I(X; X')$ es casi nula.

(c) Compresión píxel a píxel frente a compresión de residuos. En ambos esquemas Huffman queda a menos de 0.05 bits de la entropía de la fuente que codifica, de modo que el codificador no es lo que cambia: lo que cambia es la fuente. La codificación píxel a píxel trata a cada píxel como independiente de los demás y su límite es $H(X)$, que no puede aprovechar la dependencia espacial. La codificación diferencial transforma la imagen en una fuente Y cuya entropía es mucho menor que $H(X)$ cuando los vecinos están correlacionados: en `img_01` e `img_04` la pmf de los residuos se concentra alrededor de cero y la tasa de compresión pasa de 1.27 a 2.82 y de 1.12 a 1.63, con una ganancia $L_X - L_Y$ cercana a $I(X; X')$. Cuando no hay dependencia, como en `img_02`, restar dos píxeles casi independientes produce una variable con más dispersión que cada uno por separado, $H(Y) > H(X)$ y la codificación de residuos es peor que la directa. La diferencia entre ambos métodos se debe entonces a la redundancia espacial de cada imagen: la codificación diferencial gana en la medida en que $I(X; X')$ es grande y pierde cuando es cercana a cero.

6. Ejercicio 4: compresor y descompresor de línea de comando

Implementamos dos programas en Python. El compresor recibe una imagen en un formato estándar (JPG o PNG), la convierte a escala de grises, calcula los residuos sobre el recorrido en serpentina, construye el código de Huffman y escribe un archivo binario con extensión `.tp2`. El

descompresor lee ese archivo, decodifica los residuos, reconstruye la imagen por suma acumulada y la muestra en pantalla; opcionalmente la guarda y la compara píxel a píxel con la original.

El archivo contiene todo lo que el decodificador necesita y no puede deducir: una firma de 4 bytes, las dimensiones (dos enteros de 32 bits), el píxel inicial (1 byte), la cantidad de símbolos de la tabla (2 bytes), la tabla propiamente dicha (por cada residuo que aparece, su valor en 2 bytes y la longitud de su palabra de código en 1 byte), la cantidad de bits útiles (8 bytes) y, por último, un único flujo de bits con las palabras de código concatenadas en el orden de la tabla seguidas de los residuos codificados, completado con ceros hasta el byte. Guardar solo las longitudes y reconstruir el diccionario a partir del flujo de bits hace que la tabla ocupe 3 bytes por símbolo; el encabezado completo pesa alrededor de 1 KB, despreciable frente a los datos.

Tabla 5: Resultados del compresor sobre las tres imágenes. El tamaño original corresponde a la imagen en escala de grises a 8 bits por píxel; el comprimido incluye el encabezado. En los tres casos el descompresor reconstruye exactamente la imagen original.

Imagen	Bits/píxel (datos)	Original [B]	Comprimido [B]	Tasa
`img_01`	2.840	227 200	81 763	2.78
`img_02`	7.179	249 696	225 407	1.11
`img_04`	4.894	72 128	45 543	1.58

Los bits por píxel coinciden con los del Ejercicio 2 y las tasas son apenas menores por el encabezado. Verificamos con el descompresor que las tres reconstrucciones son idénticas a las originales en escala de grises. Para `img_02` el programa comprime, pero un compresor que probara ambos esquemas y guardara en el encabezado cuál usó obtendría un archivo más chico codificando las intensidades directamente.

7. Conclusiones

El código de Huffman se comportó en todos los casos como predice la teoría: la longitud media medida sobre la cadena comprimida coincidió con $L = \sum_x \hat{p}(x)\, l(x)$ y quedó entre la entropía de la fuente y la entropía más un bit, con una redundancia de solo 0.03 a 0.05 bits. Eso significa que, para una fuente dada, Huffman está prácticamente en el límite, y que la forma de comprimir más no es mejorar el codificador sino cambiar la fuente.

La codificación diferencial es exactamente ese cambio de fuente, y su efecto depende por completo de la redundancia espacial de la imagen. En la imagen suave de dunas la entropía bajó de 6.28 a 2.83 bits y la tasa de compresión subió de 1.27 a 2.82; en la partitura, de 1.12 a 1.63. En la textura de arena, en cambio, los residuos resultaron más dispersos que las intensidades y la compresión empeoró. Este último resultado nos sorprendió al principio, porque esperábamos que restar vecinos siempre ayudara; entenderlo exigió separar dos ideas que la intuición mezcla: la variabilidad de una imagen (su entropía) y su redundancia (la información mutua entre vecinos). Una imagen puede tener mucha de la primera y nada de la segunda. La información mutua fue la herramienta que ordenó el análisis: acota la ganancia posible de cualquier esquema basado en el vecino, resultó alta donde la codificación diferencial funcionó y casi nula donde falló, y la pmf conjunta permitió ver esa dependencia directamente como una diagonal más o menos nítida.

Como extensiones naturales quedan usar predictores mejores que el vecino de la izquierda (el de arriba, el promedio de ambos o el predictor de Paeth que usa PNG [7]), estimar la información mutua con vecinos más lejanos para medir cuánta redundancia queda sin explotar, y comparar con una codificación condicional que alcance $H(X \mid X')$ en lugar de $H(Y)$.

Referencias

[1] Cátedra de I206 – Inferencia y Estimación (2026). *Trabajo práctico 2: Compresión de imágenes sin pérdida mediante codificación de Huffman* y funciones auxiliares `huffman_code` y `huffman_decode`. Universidad de San Andrés.

[2] Material teórico de I206 – Inferencia y Estimación (2026): *Teoría de la información – Definiciones*. Universidad de San Andrés.

[3] Material teórico de I206 – Inferencia y Estimación (2026): *Teoría de la información – Información mutua – Fisher*. Universidad de San Andrés.

[4] Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27, 379–423 y 623–656.

[5] Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9), 1098–1101.

[6] Cover, T. M. y Thomas, J. A. (2006). *Elements of Information Theory* (segunda edición). Wiley. Capítulos 2 y 5.

[7] Boutell, T. et al. (1997). *PNG (Portable Network Graphics) Specification*, RFC 2083, sección 6: filtros y predictor de Paeth.